Assignment 04 Recursion

Due Date: April 03, 2026

Assignment Objective

To solve a maze recursively!

1 The Maze Definition

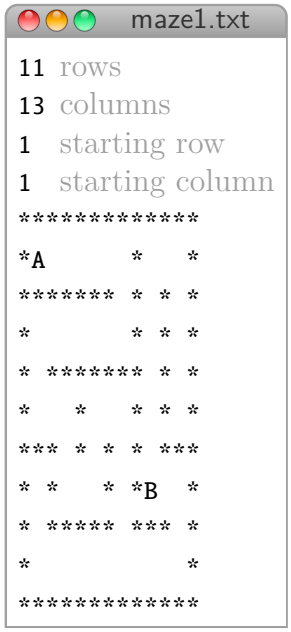
We will be solving a maze recursively. A maze is read from a `.txt` file using `ASCIIDataFile`. Hardcode the filename as such for testing:

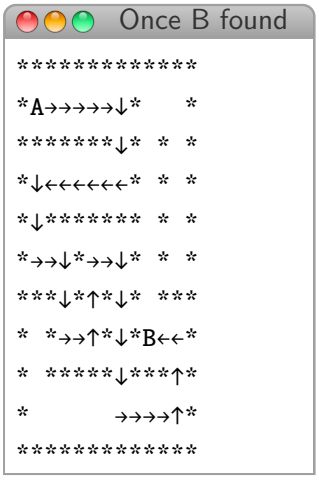
```
ASCIIDataFile file = new ASCIIDataFile("maze1.txt");
```

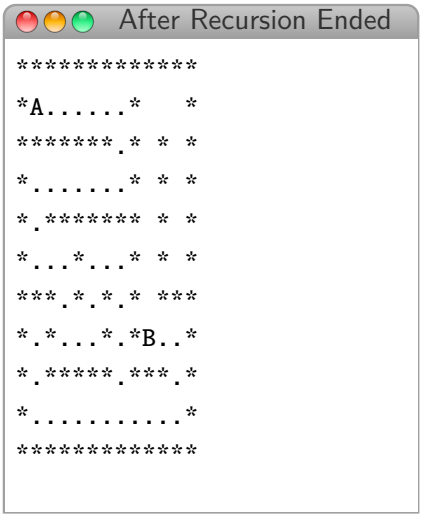
It will be stored inside of a 2-D array of `char`acters. The size will be given in the file. The file also contains the starting position, which is the cell containing `A` and we will try to recursively end at `B`. There will be four directions to examine at each cell: right, left, down and up. By default, a maze file will asterisks `*` to denote walls (a dead end), and spaces, which we will denote as `␣`, denoting a valid cell to enter. Then, we recursively try to find the target cell `B`. Not all cells will yield a path to the ending cell. Hence, if we encounter a cell that leads to a dead end (remember that recursion will try all possible paths and will backtrack/pop to go back to previous coordinates), we need mark it as "it has been visited and will lead to a dead end, don't enter it again". We will use a period character `.` for this. That way, a period will be treated as a `*` (*i.e.*, wall). Here are the different characters we have discussed and will discuss later:

Character	Meaning	Will you manually place it?
`*`	Wall	No
`␣`	Valid cell to enter	No
`A`	Starting cell	No
`B`	Target cell	No
`.`	Visited cell that leads to a dead end	Yes
`→`	Path taken to the right	Yes
`←`	Path taken to the left	Yes
`↑`	Path taken upwards	Yes
`↓`	Path taken downwards	Yes

A simple maze is shown next. Once the program works on it, the program should also work on the bigger maze. The solution is shown as well.







The middle window is the state of the maze as soon as the target cell (*i.e.*, `B`) has been found. The window on the right is the maze after all the recursion calls have been completed.

1.1 The `Direction` Enum

In case you are not comfortable with `enum`s, please have a look at section 2. The `Direction` `enum` will contain five values: the four possible directions and the value `None`, which denotes a dead end. This is a three-line code snippet, don't overthink it (*hint*, the same skeleton found in section 2 can be used here, but *not* the `TestSeason` as `TestSeason` is there to show you how to use the `enum`).

1.2 The `print` Method

The `print` method will accept a 2-D array of `char`acters and print its content on the screen. Please differentiate between `System.out.print(...);` versus `System.out.println(...);`. Also, please don't hardcode the dimensions of the maze, I faith that you will properly complete this. Sometimes, IntelliJ might remove consecutive spaces in the output when printing to the console, so be aware of this. In case the logic is done properly, you should see a rectangular maze, not a ragged one. We want a font that is monospaced, which means all characters have the same width (*e.g.*, a period and uppercase W will have the same width `.w` versus .W). Email me (`amaysara@brocku.ca`) if you have issues with this.

1.3 The `insert` Method

The `insert` method is a `void` method that will accept the coordinate of the cell and a direction. It will insert the corresponding arrow in the cell based on the direction. For example, in case the direction is right, it will insert → in the cell. In case the direction is

left, it will insert ← in the cell, the same for the up and down. Lastly, in case the direction is none, it will insert a period `.` in the cell. Ensure to **not** override the start and end cells so that `A` and `B` always exist. To write the arrows in Java, use the following code snippet:

```
//The 0x____ means it is a Unicode character.
char left = 0x2190;  // ←
char right = 0x2192; // →
char up = 0x2191;    // ↑
char down = 0x2193;  // ↓
```

1.4 The `readFile` Method

A sample maze map has been added to the right. The first four `int`eger reads **MUST** be done in the constructor. They are, in order:

- The number of rows (one-based), which is the 11 in the maze file,
- The number of columns (one-based), which is the 13 in the maze file,
- The starting row (zero-based), which is the 1 in the maze file,
- The starting column (zero-based), which is the 1 in the maze file.
- The starting coordinate will always be $(1, 1)$ in this assignment, but you must always read it from the file to ensure your code works on any maze file, beyond the mazes provided for this assignment.
- The maze itself (read within `readFile`), which is a grid of characters rows × columns.

maze.txt

```
11
13
1
1
*************
*A      *   *
******* * * *
*       * * *
* ******* * *
*   *   * * *
*** * * * ***
* *   * *B  *
* ***** *** *
*           *
*************
```

The first two values will be used to create the 2-D array, while the last two values will be used as the starting coordinates for the recursive method. Ensure to read the first four `int`egers *outside* and before the loop, then, use nested `for`-loops to read and store the maze in the 2-D array. No `while` loops will be used. Also, don't forget to use `file.setDelay(0);` to speed up the reading process and `file.close();` after the `for`-loops.

1.5 The `solve` Method

The `solve` method is a `void` method that will recursively solve the maze. What should it accept as parameters? Recall that parameters are see as 'mini memory' for any recursive method. Also, **NEVER** override/reinitialize the parameter values. In case needed (which is not needed in this assignment but for future reference), create a local variables and assign them the updated values, but never modify the parameters directly.

Moreover, it will keep track of the number of recursive iterations required to reach the target (*i.e.*, once `B` is found, where we still have activation records in the call stack and haven't popped everything yet).

Hint: this method needs to start somewhere in the grid and keep track of the number of iterations. Figure out the 'starting somewhere in the grid' part and by default, we have `0` iterations. These will be your parameters.

It should be noted that only the 2-D array storing the maze needs to be an instance variable. No other instance variables are allowed to be created and will earn you a grade of zero if there is another instance variable (the number of iterations is *not* an instance variable 😉).

We need to understand that base cases might be direct or indirect depending on our conditional statements. For instance, consider the following recursive method:

```
private void recursion(){
  if(0 == 1){
    recursion();
  }
}
```

The above recursive method will not run, even though a recursive call exist and no *explicit* base case is found. It would be more suitable to explicitly include all base cases.

In our case, we need to start at cell `A` and end at cell `B` where both of these cells are not the space character but valid cells to enter. Recall that cell `A` and `B` must **not** be overridden throughout the execution (the overridding was discussed previously).

Once we reach cell `B`, we are done with recursion (this sentence is a hint).

When we reach cell `B`, we need to print out the current maze and the number of iterations. *Again*, the number of iterations is not an instance variable 😉.

Moreover, in our case (pun not intended), it is best to use `if`–`else if`–`else if`–...–`else` logic. Each of the following characters need to be dealt with: `A`, `B`, `␣`, `.` and `*`. You are welcome to use `else{ return; }` to ensure the current call ends as Java might complain.

To ensure you are properly programming this, you will always check the **current** cell you are in and then decide what to do. For example, you will enter a block cell (*e.g.*, `*` or `.`) and check if the current cell is a block cell or a space, or etc. In other words, don't 'look ahead' and say 'from the current cell, I will look at the right cell and check if it is a space or not'. Instead, enter the cell and figure out what to do within it. All mazes are designed so that you don't have `IndexOutOfBoundsException` since all of the boundary cells are `*`.

You might be wondering when will we be checking the arrows →, ←, ↑ and ↓? We will not! This is because they will be overwritten with `.` eventually. Here is how the logic works:

- **The overview**: In case the current cell is a space, change current cell to → and perform recursive call going to the right, left, down and then up.
- Thus, we first go to the right cell, and eventually, we might go back to perform the left recursive call.
- Even though we marked the current cell with →, we will realize that going to the right didn't work, let us try going to the left.
- Since the current cell is marked as →, we change the current cell to ←, and then perform the recursive call of going to the left.
- The logic continues on if we get back to go down and then up.

The summary is that if the current cell is a space, we will:

★ The heart of recursion

mark current cell with →	and perform recursive call going to the right
mark current cell with ←	and perform recursive call going to the left
mark current cell with ↓	and perform recursive call going downwards
mark current cell with ↑	and perform recursive call going upwards
mark current cell with `.`	because all directions have been explored

In case we examined all of the directions and we still had to come back, it means that this cell doesn't yield to the target. Mark it as a period `.` and don't perform any recursive calls after that. In case the maze is not solvable, then periods should be placed in all of the accessible spaces which is done automatically by the recursive calls. Some spaces are not accessible, hence, they cannot be visited and thus, are not marked as `.`. In case the directions are swapped, say we go up first, then down, then left and then right, the program will still work, but the path taken will be different. It doesn't matter which direction we go first, as long as the logic is correct.

2 What are Enums?

Suppose that we would like to store the four seasons in a year, which are: fall, winter, spring, and summer. Which Java datatype should we use: `int`eger, `double`, `String`, `boolean` or something else? Well, we cannot use `boolean` as it only stores two values: `true` and `false`. We *could* use an `int`eger, but it allows use to store 4.28 billion values, which is not what we want. We *could* also use a `String`, but it also allow us to store any string, which is also not what we want. Can we create our own datatype that only allows us to store *exactly* four values of the season? Yes, we can with the usage of an `enum`! We can use an `enum` to create a custom datatype that has our custom values. Create a new file named `Season.java` and write the following code, where the values are comma-separated and all uppercase by convention (or using `MY_LONG_VALUE` for multi-word values):

```
// Only three lines of code are needed to create an enum!
public enum Season {
  FALL, WINTER, SPRING, SUMMER
}
```

```
public class TestSeason { // The test class named TestSeason.java showing how to use the Season enum.
  public TestSeason() {
    Season myFavoriteSeason = Season.SUMMER;
    printSeason(myFavoriteSeason);
    Season currentSeason = getCurrentSeason();
    System.out.println("Current season is: " + currentSeason);
  }
  private void printSeason(Season s) {// Accepting a Season enum as a parameter.
    if(s == Season.FALL){
      System.out.println("It's fall!");
    } else if(s == Season.WINTER){
      System.out.println("It's winter!");
    } else if(s == Season.SPRING){
      System.out.println("It's spring!");
    } else if(s == Season.SUMMER){
      System.out.println("It's summer!");
    } else {
      System.out.println("Invalid season."); // when s is null.
    }
  }
  private Season getCurrentSeason() { // Having a Season enum as a return type.
    return Season.WINTER;
  }
  public static void main(String[] args) {
    TestSeason ts = new TestSeason();
  }
}
//Output:
//It's summer!
//Current season is: WINTER
```

3 Program Execution

There are six different mazes to test your program on. The mazes in `maze1.txt`, `maze2.txt` and `maze3.txt` are solvable, while the mazes in `maze1u.txt`, `maze2u.txt` and `maze3u.txt` are unsolvable ('u' stands for unsolvable). The output of all mazes is found next. Again, the order of the directions doesn't matter, so your path taken might be different than the one shown here. Different direction order might generate different number of steps, which is Ok.

maze1.txt (Solvable)

```
11 rows
13 columns
1  starting row
1  starting column
**************
*A       *   *
******* * * *
*        * * *
* ******* * *
*   *    * * *
*** * * * ***
* *   * *B  *
* ***** *** *
*           *
**************
```

Output (Solvable)

```
Found B with 38 steps!
**************
*A→→→→→↓*   *
*******↓* * *
*↓←←←←←←* * *
*↓******* * *
*→→↓*→→↓* * *
***↓*↑**↓* ***
* *→→↑*↓*B←←*
* *****↓***↑**
*      →→→→↑*
**************

Printing the maze:

**************
*A......*   *
*******.* * *
*.......* * *
*.******* * *
*...*...* * *
***.*.*.* ***
*.*...*.*B..*
*.*****.***.*
*...........*
**************
```

maze1u.txt (Unsolvable)

```
11 rows
13 columns
1  starting row
1  starting column
**************
*A*      *   *
* *** * * * *
*      *   * *
************ *
*        *   *
* ***** *****
*      * *B* *
* *** ***** *
*   *        *
**************
```

Output (Unsolvable)

```
Printing the maze:
**************
*A*.....*...*
*.***.*.*.*.*
*.....*...*.*
***********.*
*        *...*
* ***** *****
*      * *B* *
* *** ***** *
*   *        *
**************
```

maze2.txt (Solvable)

```
15 rows
19 columns
1  starting row
1  starting column
*********************
*A  *     *     * *
*** * *** * * * * *
* * *   * * * *   *
* * *** *** * *****
* *   *     * *   *
* *** * ***** * * *
* *   *   * *   * *
* * ***** * ***** *
* *     *     *   *
* ***** ******* * *
*   *   *     * * *
* *** *** *** * * *
*         *  B  * *
*********************
```

Output (Solvable)

```
Found B with 36 steps!
*********************
*A→↓*     *     * *
***↓* *** * * * * *
* *↓*   * * * *   *
* *↓*** *** * *****
* *→→↓*     * *   *
* ***↓* ***** * * *
* *↓←←*   * *   * *
* *↓***** * ***** *
* *→→→→↓*     *   *
* *****↓******* * *
*   *↓←←*→→→→↓* * *
* ***↓***↑***↓* * *
*    →→→→↑*  B  * *
*********************

Printing the maze:

*********************
*A..*     *     * *
***.* *** * * * * *
*.*.*   * * * *   *
*.*.*** *** * *****
*.*...*     * *   *
*.***.* ***** * * *
*.*...*   * *   * *
*.*.***** * ***** *
*.*.....*     *   *
*.*****.******* * *
*...*...*.....* * *
*.***.***.***.* * *
*.........*  B  * *
*********************
```

maze2u.txt (Unsolvable)

```
15 rows
19 columns
1  starting row
1  starting column
*********************
*A  *     *     * *
*** * *** * * * * *
* * *   * * * *   *
* * *** *** * *****
* *   *     * *   *
* *** * ***** * * *
* *   *   * *   * *
* * ***** * ***** *
* *     *     *   *
* ***** ******* * *
*   *   *     * * *
* *** *** *** * * *
*         *  B  * *
*********************
```

Output (Unsolvable)

```
Printing the maze:


*********************
*A.........*.....*
**********.*.***.*
*.....*.*...*.*.*.*
*.*.*.*.*.***.*.*.*
*.*.*...*.....*.*.*
***.***.*******.*.*
*...*...*...*...*.*
*.*.*****.*.*.*.*.*
*.*.*.....*.*.*...*
*.*.*.*****.*.*****
*.*.*.....*...*   *
*.*******.***** * *
*..........*B* * *
*********************
```

maze3.txt (Solvable)

```
31 rows
31 columns
1  starting row
1  starting column
**********************************
*A  *           *           *      *
*** * ***** * * ***** * * *** *
*   *   * * * * *     * *   * *
* ***** * * * *** ***** *** * *
* *       * *     *   *   * * *
* * ***** * ******* * *** *** *
* * *   * *     *   *   *     *
* * * * ***** * *** **********
* * * *     * *   *       *   *
* *** ***** ***** * *** * * ***
*   *     * *     *   * * *   *
*** *** * * * ********** * *** *
* *   * * *   *         * *   *
* *** *** ***** ******* *** ***
*   * *   * *       *   *   * *
* *** * *** * ***** *** * *** *
* *   * *         *   * * *   *
* * *** ************* *** *** *
* * *   *       *     *   *   *
* * * * ***** * *** * *** ***
* * * * * *     *   * *   *   *
* * * * * ******* ***** *** *
* * * *   * *     * *   *   * *
* * ******* * *** * * * *** * *
*   *     * * *   * * *   * * *
* *** *** * * * *** * *** * * *
*   * *   *   *   * * *   * * *
*** * * ********* * * * *** * *
*     *           *   *      B*
**********************************
```

Output (Solvable)

```
Found B with 240 steps!
**********************************
*A→↓*     →→↓*     →→↓*     *
***↓* *****↑*↓* *****↑*↓* *** *
*↓←←*   * *↑*↓* *→→→→↑*↓*   * *
*↓***** * *↑*↓***↑*****↓*** * *
*↓*       *↑*→→→→↑*   *→→↓* * *
*↓* ***** *↑******* * ***↓*** *
*↓* *→→↓* *↑←←←←*   *   *→→→→↓*
*↓* *↑*↓***** *↑*** **********↓*
*↓* *↑*→→→→↓* *↑←←*       *↓←←*
*↓***↑*****↓*****↑* *** * *↓***
*→→↓*↑←←←←*↓*→→→→↑*   * * *→→↓*
***↓*** *↑*↓*↑********** * ***↓*
* *→→↓* *↑*→→↑*         * *↓←←*
* ***↓***↑***** ******* ***↓***
*   *↓*→→↑* *       *   *↓←←* *
* ***↓*↑*** * ***** *** *↓*** *
* *↓←←*↑*          *   * *↓*   *
* *↓***↑************** ***↓*** *
* *↓*..↑*↓←←←←←←*     *↓←←*   *
* *↓*.*↑*↓*****↑* *** *↓*** ***
* *↓*.*↑*↓*→→→→↑*   * *→→↓*   *
* *↓*.*↑*↓*↑******* *****↓*** *
* *↓*.*↑←←*↑*↓←←←←* *   *→→↓* *
* *↓********↑*↓***↑* * * ***↓* *
*↓←←*→→→→↓*↑*↓*→→↑* * *   *↓* *
*↓***↑***↓*↑*↓*↑*** * *** *↓* *
*→→↓*↑*↓←←*↑←←*↑←←* * *   *↓* *
***↓*↑*↓**********↑* * * ***↓* *
*  →→↑*→→→→→→→→→→↑*   *    →→B*
**********************************

Printing the maze:

**********************************
*A..*.........*.........*.....*
***.*.*****.*.*.*****.*.*.***.*
*...*...*.*.*.*.....*.*...*.*
*.*****.*.*.*.***.******.***.*.*
*.*.......*.*.....*...*...*.*.*
*.*.*****.*.*******.*.***.***.*
*.*.*...*.*.....*...*...*.....*
*.*.*.*.*****.*.***.*********.*
*.*.*.*.....*.*...*.......*...*
*.***.*****.*****.*.***.*.*.***
*...*.....*.*.....*...*.*.*...*
***.***.*.*.*.*********.*.***.*
*.*...*.*.*...*.........*.*...*
*.***.***.*****.*******.***.***
*...*.*...*.*.......*...*...* *
*.***.*.***.*.*****.***.*.*** *
*.*...*.*.........*...*.*.*   *
*.*.***.*************.***.*** *
*.*.*...*.......*.....*...*   *
*.*.*.*.*.*****.*.***.*.*** ***
*.*.*.*.*.*.....*...*.*...*   *
*.*.*.*.*.*.*******.*****.*** *
*.*.*.*...*.*.....*.*...*...* *
*.*.*******.*.***.*.*.*.***.* *
*...*.....*.*.*...*.*.*...*.* *
*.***.***.*.*.*.***.*.***.*.* *
*...*.*...*...*...*.*.*...*.* *
***.*.*.*********.*.*.*.***.* *
*.....*...........*...*......B*
**********************************
```

maze3u.txt (Unsolvable)

```
31 rows
31 columns
1  starting row
1  starting column
*********************************
*A  *         *         *     *
*** * ***** * * ***** * * *** *
*   *   * * * * *     * *   * *
* ***** * * * *** ***** *** * *
* *       * *     *   *   * * *
* * ***** * ******* * *** *** *
* * *   * *     *   *   *     *
* * * * ***** * *** ********* *
* * * *     * *   *       *   *
* *** ***** ***** * *** * * ***
*   *     * *     *   * * *   *
*** *** * * * ********* * *** *
* *   * * *   *         * *   *
* *** *** ***** ******* *** ***
*   * *   * *       *   *   * *
* *** * *** * ***** *** * *** *
* *   * *         *   * * *   *
* * *** ************* *** *** *
* * *   *       *     *   *   *
* * * * * ***** * *** * *** ***
* * * * * *     *   * *   *   *
* * * * * * ******* ***** *** *
* * * *   * *     * *   *   * *
* * ******* * *** * * * *** * *
*   *     * * *   * * *   * * *
* *** *** * * * *** * *** * * *
*   * *   *   *   * * *   * * *
*** * * ********* * * * *** * *
*     *           *   *      B*
*********************************
```

Output (Unsolvable)

```
Printing the maze:

*********************************
*A*.......*...........*.......*
*.*.*******.*.*******.*.*****.*
*.*.....*...*.*...*...*...*...*
*.*****.*.***.*.*.*.*****.*.***
*.....*...*...*.*.*...*...*.*.*
*****.*****.***.*.***.*.***.*.*
*.*...*.....*.*.*...*...*...*.*
*.*.***.*****.*.*.*.*****.***.*
*.*.*.....*...*.*.*.....*.*...*
*.*.*****.*.***.*.*******.*.*.*
*.*...*...*...*.*.........*.*.*
*.***.*.*****.*.*************.*
*...*...*.....*...*...*.....*.*
*.*.*****.*.*****.*.***.*.*.*.*
*.*.....*.*.......*.....*.*.*.*
*.***.*.*.***********.***.*.*.*
*.*.*.*.*...........*.*...*.*.*
*.*.*.*************.***.***.*.*
*.*.*.*.*******...*.....*.*.*.*
*.*.*.*.*.***.***.*******.*.*.*
*.*.*.*.*...*...*.....*...*...*
*.*.*.*.***.***.*.***.*.*****.*
*.*.*...*.*.*.*...*...*.....*.*
*.*.*****.*.*.*****.*******.*.*
*.....*...*.*.......*.......*.*
*****.*.***.*******.***.*****.*
*...*.*...*.*.....*.....*.....*
*.***.***.*.*.***.*******.*****
*.........*.....*...........*B*
*********************************
```

For submission, ensure `maze3.txt` is hardcoded in the program so that the TA can execute it and place in all of the six maze files in the project.

Pseudocode Explanation and Examples

Pseudocode is a high-level description of an algorithm or a method that allows the writing of the logic in a way that is easily understandable as opposed to using Java syntax or any other programming language syntax. It is the rough idea of how the code will be structured which includes the important steps needed. It is important to plan out the solution before jumping into coding. Thus, we need to create pseudocode for each method mentioned in the assignment. The pseudocode needs to be detailed enough to cover all of the major steps needed to implement the method. Here is a sample (by a sample, it means other breakdowns are also correct) of pseudocode for a method that calculates the sum of an array of `int`egers:

```
getArraySum(int[] array) → int
  total ← 0
  n ← array length
  loop i = 0 to < n {
    total = total + array[i]
  }
  return total
```

Here is another example of a method that prints a 2D integer array:

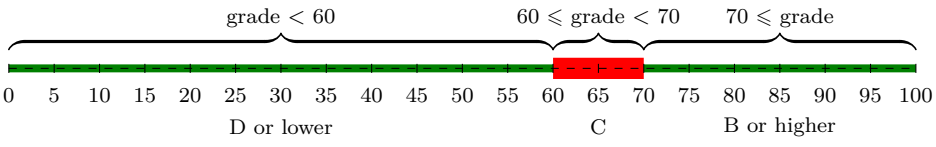
```
print2DArray(int[][] array) → void
  n ← array length
  loop r = 0 to < n {
    loop c = 0 to < array[r] length {
      print(array[r][c])
    }
    new line
  }
```

Next, the pseudocode for reading from a file until the end of the file is reached:

```
while true {
  name = file readString
  if file is EOF → break
  age = file readInt
  height = file readDouble
  print(name, age, height)
}
```

Lastly, Applying De Morgan's Law and its rough diagram representation:

```
grade ← randomInt(0, 100)
if !( 60 ⩽ grade and grade < 70) {
  print("grade is not C")
} else{
  print("grade is C")
}
```



Code Documentation

We need to properly document the code using Java's documentation standards. For each method in the class(es), each parameter and return type need to be documented along with a description of the method and the significance of each parameter and return type. Here is an example of documenting a function method:

```
/**
 * Calculates the multiplication of the two parameters and return the
 * multiplication value.
 *
 * @param x the left operand to multiply.
 * @param y the right operand to multiply.
 * @return the result of multiplying the two parameters.
 */
private int multiply(int x, int y){
  return x * y;
}
```

In case the method accepts one parameter, then `@param` is used once. In case the method doesn't accept any parameters, then `@param` is not used. In case the method is a `void` method (*i.e.*, *not* a function method), then `@return` is not used.

In addition, we need to add a class header, which is a comment block after the `import` statements, but before creating the `class`. It will be something like this:

```
import brock.*;

/**
 * This is a simple class that will show how to insert the class
 * header (this block) and comment the methods in the class.
 *
 * @author     Maysara Al Jumaily (please change to your name)
 * @course     COSC 1P03
 * @assignment #4
 * @id         1234567 (please change to your student id)
 * @version    1.0
 * @since      January 01st, 2026
 */
public class Test {
  ...
}
```

Submission

Submit a `.zip` (**NOT** a `.rar` file or `.7z`) file on Brightspace containing two folders:

- The first folder is the entire IntelliJ project folder, which contains the out and src folders along with other files created by IntelliJ. Ensure the `.java` files are present in the `src` folder.
- The `pseudocode` folder contains the pseudocode for each method in the assignment **must be less than 10mb in size** uncompressed (Brock JAR file must be included). Scale the images down if you have large images, watch this for Windows 11 and macOS. It should be readable after compression! It could be at least one file or multiple files depending on how you created the pseudocode. Make sure the files are either `.jpg`/`.jpeg`, `.png` or `.pdf`.
  - Make sure the files are either `.jpg`/`.jpeg`, `.png` or `.pdf`. Please have a pattern for naming the files, such as `00.png`, `01.png`, `02.png`, etc. or `pseudocode_01.pdf`, `pseudocode_02.pdf`, etc.

Now, to create the `.zip` file, follow the steps below depending on your operating system:

- To zip a file on **Windows**:
  - Right-click on the folder → Send to → Compressed (zipped) folder, or just Zip folder.
  - Rename that zipped folder to `a04-9876543`, where `9876543` is your student number.
- To zip a file on **macOS**:
  - Right-click (or double tap) on the folder and then choose Compress "...".
  - Rename that zipped folder to `a04-9876543`, where `9876543` is your student number.
- **For the cautious**: to ensure everything is done properly, copy the zipped folder to another location (*e.g.*, `documents` or `desktop` or `downloads`, etc.) and unzip it.
  - **On windows**: right-click on it → Extract All... → keep the setting as is and click on the **Extract** button. It should open the folder containing all of the files.
  - **On macOS**: double-click on the `.zip` folder to automatically exact the content and open the newly generated folder.
  - Open IntelliJ and open the project by going to File → Open... and navigate to the unzipped folder and select it. It should open the project without any issues.
- **Failure to submit the pseudocode yields a grade of zero!**
- **Failure to not PRECISELY follow the steps above yields a grade of zero!**